Supporting Data and Code: Reproducibility Documentation

Evaluating LLM-Based Credit Rating Systems: A Pre-Registered Specification-Curve Analysis

Companion to the manuscript submission

1. Purpose and scope

This document describes the supporting data-and-code archive that accompanies the manuscript. The archive is a self-contained, **offline, deterministic** reproduction: from a single command it regenerates every reported number, table, and figure from frozen inputs, with **no network access, no API keys, and no paid model calls**. The permanent citable version is the Zenodo archive at `10.5281/zenodo.21953934` (code MIT-licensed); the same tree is mirrored at `https://github.com/samirrc2/llm-credit-rating-specification-curve`.

A strict wall separates two phases. An *offline analysis* reproduces every claim from frozen inputs (documented here). A *live capture* phase originally queried commercial model APIs and is retained for transparency only (Section 8). Because commercial models change over time, capture is deliberately not part of the reproducible core; the frozen model-response corpora it produced are shipped and hash-verified. The confirmatory data are synthetic firm profiles benchmarked to an objective Altman Z″ reference; the real-firm arm uses only publicly available SEC filings, anonymized and perturbed.

2. Quick start

```
pipinstall-rrequirements.txt
bashreproduce.sh
```

`reproduce.sh` invokes `python3` unless the environment variable `PY` names an interpreter, so run `PY=/path/to/pythonbashreproduce.sh` when the `python3` on `PATH` is not the environment built from `requirements.txt`. An Anaconda base environment, for instance, aborts in `analysis/run_analysis.py` with `ImportError:cannotimportname'_lazywhere'fromscipy._lib._util`, because its `scipy` and `statsmodels` disagree. **Expected result:** the full analysis regenerates every `results/results_*.json`; a second independent run is byte-identical (the SHA-256 of each JSON result matches), and the run exits 0. Total runtime is a few minutes on a standard laptop. No network access is required at any point.

3. Archive layout

`reproduce.sh`	one-command offline reproduction of all results
`requirements.txt`	pinned Python dependencies (analysis only; Python 3.10)
`config/`	specification grid, paraphrase templates, rating scale, run config
`analysis/`	all analysis code (Section 5)

capture/	live data-collection harness, not run here (Section 8). Three paths under it are nonetheless reproduction inputs: the two frozen corpora below, and freeze_deco ntam.py, imported by the decontamination analysis
data/frozen/main/	pre-registered inputs: battery_90.json, grid_definiti on.json, paraphrase_templates.json, rating_scale.js on, capital_map.json, manifests
data/frozen/realarm/	real-firm arm artifacts: anonymized battery, sealed crosswalk, rating provenance, 12-spec grid
data/frozen/market_data/	FRED option-adjusted-spread series (cached)
data/raw/main/	frozen response corpus (per-call JSON)
data/raw/realarm/	real-firm arm model-output panels (arm / comparator / fingerprint)
capture/flagship_runs/	flagship model-tier arm corpus (1,620 per-call JSON), fixed by MANIFEST_FLAGSHIP
capture/reasoning_runs/	reasoning-architecture arm corpus (DeepSeek-R1; 1,620 per-call JSON + consolidated panel), fixed by MA NIFEST_REASONING
data/panel/panel.parquet	analysis panel, built once from raw (the confirmatory analysis input)
results/	generated outputs (Section 6), with one exception: the four SPEC_*.sha256 files are pre-registration hashes, committed before computation and never written by any script
paper/NLP/	manuscript (Elsevier CAS), figures, highlights, cover letter, .docx
manifest/	SHA-256 manifests for every frozen stage
docs/	reports, appendix, changelog
PREREGISTRATION.md, PREREGISTRA TION_AMENDMENTS.md	frozen pre-registration and amendments

4. Data (inputs and metadata)

4.1 Frozen inputs

Every input the pipeline reads is a frozen artifact. The analysis is a pure, seeded function of these files.

File	Rows	Unit of observation / key fields
data/frozen/main/battery_ 90.json	90	one synthetic firm profile; 45 credit-health items (SAFE/WATCH/DISTRESS, 15 each) with an Altman Z″ benchmark band + 45 directional items (buy/hold/sell). Fields: item_ id, family, benchmark_label, facts_kv, facts_t ext

File	Rows	Unit of observation / key fields
data/raw/main/run_20260707_185649/	12,960	one row = one model call across the pre-registered grid; the **confirmatory** analysis uses the two providers with complete current+prior coverage (OpenAI, Google) = $90 \times 32 \times 3 = 8{,}640$ cells. An excluded third-provider pilot is retained for transparency. Fields: spec axes, item_id, seed, provider, model, raw_response
data/frozen/realarm/battery_real45.json	31	anonymized, perturbed real B2B issuers with disclosed agency ratings; benchmarked in ratings_provenance.csv (verbatim 10-K quote, SEC URL, document SHA-256)
data/frozen/realarm/spec_grid_12.json	12	the frozen 12-specification grid (presentation × provider × paraphrase × format) shared by the real-firm and flagship arms
capture/flagship_runs/run_20260830_141844/	1,620	flagship model-tier arm: gpt-5.4 + gemini-3.1-pro-preview on the 12-spec grid × 45 credit items × 3 seeds
capture/reasoning_runs/run_20260906_111300/	1,620	reasoning-architecture arm: deepseek-r1 on the 12-spec grid × 45 credit items × 3 seeds (reasoning_panel.json is the consolidated input read by reasoning_analysis.py)
config/grid_definition.json	—	the balance-constrained D-optimal specification grid
data/frozen/main/rating_scale.json	—	21-grade letter scale and the letter→notch/band/IG-HY maps
data/frozen/main/capital_map.json	—	Basel CRE20 ECRA risk-weight map (retained for provenance; the post-hoc capital analysis was removed in the major revision and is no longer part of the pipeline)

4.2 Provenance and integrity

manifest/ holds a SHA-256 manifest for every frozen stage: MANIFEST_INPUTS (pre-registered inputs), MANIFEST_RAW (response corpus), MANIFEST_PANEL (analysis panel), MANIFEST_ANALYSIS, MANIFEST_CAPITAL, MANIFEST_MARKET, MANIFEST_REALARM, and MANIFEST_FLAGSHIP (flagship arm). PREREGISTRATION.md and PREREGISTRATION_AMENDMENTS.md freeze the design, so any silent change to an input or to the pre-registered plan is detectable before the analysis runs. Post-hoc analyses (the flagship tier arm, the self-consistency curve, and the score validation) are labelled as such in both the manuscript and this document.

4.3 Data sources

All confirmatory inputs are synthetic and self-contained: the 90-item battery is frozen and its Altman Z″ benchmark is independently computable before any model is queried. The real-firm arm uses only public SEC 10-K filings and XBRL company facts (keyless EDGAR; permanent URLs in ratings_provenance.csv); the disposable build cache is not shipped and is not needed to reproduce any result. The original capture queried the OpenAI and Google APIs; those

responses are frozen under `data/raw/` and `capture/flagship_runs/` and are the only trace of the live phase the reproduction consumes.

5. Code (what it consumes and runs)

5.1 Analysis scripts

All code is in `analysis/`; each module is deterministic (seed 42) and reads only frozen data.

Script	Purpose	Produces
`run_analysis.py`	confirmatory analysis: OLS Type-II ANOVA variance decomposition (item / axes / provider×item / pure-seed / other), within/cross-vendor agreement, granularity κ, IG/HY flip	`results.json`, `spec_curves.json`, `MANIFEST_ANALYSIS`
`addenda.py`	supplementary robustness figures/numbers	`results_addenda.json`
`benchmark_validation.py`	power / benchmark validation checks	`benchmark_validation.*`, `MANIFEST_VALIDATION`
`analyze_realarm.py`	real-firm arm: fingerprint gate, WATCH-stratum flip-share, descriptive arm-vs-battery comparison (no equivalence test), band accuracy vs agency	`H1_results.json`
`realarm_speccurve.py`	external-benchmark specification curve on real issuers (agency ratings; issuer-preserving permutation)	`results_realarm_speccurve_*.json`
`realarm_scale_sensitivity.py`	does instability track absolute dollar scale? (Z''-invariance, Spearman)	`results_scale_sensitivity.json`
`tier_analysis.py`	flagship vs nano/flash IG/HY & band flip on the 12-spec grid, issuer-clustered CIs	`results_tier.json`
`extended_analysis.py`	specification-instability score validation (ROC-AUC, held-out & real-issuer), self-consistency curve, ordinal-weighted agreement, per-stratum flips, deployment cost	`results_extended.json`
`revision_analysis.py`	boundary-distance severity (R3-7) and turnover recast (R3-6)	`results_revision.json`
`permtest_and_tau.py`	rating-invariance permutation null (R3-3) and SIS τ calibration (R3-5)	`results_permtest_tau.json`
`variance_model_range.py`	range of variance explained by elicitation axes (main-effects / +2-way / issuer-OOS ridge)	`results_variance_model_range.json`
`crossmodel_aliasing.py`	cross-model transfer of the instability score and design aliasing diagnostics (R2-4/R2-5/R3-2)	`results_crossmodel_aliasing.json`
`reasoning_analysis.py`	reasoning-architecture arm (DeepSeek-R1) vs nano/flash & flagship on the frozen grid	`results_reasoning.json`

Script	Purpose	Produces
`build_panel.py`	(audit) rebuild `panel.parquet` from the raw corpus	panel (row-identical)
`make_figures.py`, `make_extended_figures.py`	regenerate manuscript figures (optional; needs matplotlib)	`paper/NLP/*.pdf,png`

5.2 Pipeline order

`reproduce.sh` runs the modules in dependency order; all JSON outputs are timestamp-free, so re-runs are byte-identical:

```
run_analysis→addenda→benchmark_validation→analyze_realarm→realarm_speccu
rve→realarm_scale_sensitivity→tier_analysis→extended_analysis→revision_a
nalysis→permtest_and_tau→variance_model_range→crossmodel_aliasing→reason
ing_analysis
```

6. Results (generated outputs)

Running the pipeline regenerates the following under `results/`. In a clean checkout these already exist; the reproduction overwrites them and confirms they are unchanged.

Output	Contents
`results.json`	headline confirmatory numbers: variance shares (with issuer-clustered CIs), agreement ladder (within 82.5% / cross 64.4%), IG/HY matched-pair disagreement (25.2%)
`spec_curves.json`	per-specification label-recovery / IG-share for the specification-curve figure
`results_permtest_tau.json`	rating-invariance permutation null (temperature-0-honored subgrid: observed 1.59, $p = 0.0005$, $\eta^2 = 0.46$) and SIS τ calibration ($\tau = 0.08$)
`results_tier.json`	flagship vs nano/flash flip shares and paired differences (-1.1 pp, 95% CI -7.6 to $+6.0$)
`results_reasoning.json`	DeepSeek-R1 reasoning arm: IG/HY flip 25.5% (95% CI 20.9–30.1), paired diff vs nano/flash +0.9 pp
`results_extended.json`	score validation (held-out ROC-AUC 0.94, CI 0.86–0.99; real-issuer 0.70), self-consistency curve ($k = 1 \ldots 15$), Krippendorff α, per-stratum flips, deployment cost
`results_crossmodel_aliasing.json`	cross-model transfer (Spearman 0.14–0.73, median 0.45; AUC 0.72) and aliasing (main off-diag 0.062; main×2FI 0.333)
`results_variance_model_range.json`	variance explained by elicitation axes: 0.083 / 0.119 / 0.101 (main / +2-way / issuer-OOS)
`results_realarm_speccurve_*.json`	external-benchmark spec curve on real issuers (42.2–61.1%, mean 53.7%, perm $p = 0.021$)
`results_scale_sensitivity.json`	scale-invariance check (Spearman -0.035, perm $p = 0.85$)
`results_revision.json`, `results_addenda.json`	boundary-distance severity, turnover recast, supplementary robustness

Output	Contents
`H1_results.json`	real-firm arm descriptive outputs (WATCH flip-shares, arm−battery difference, band accuracy)
`benchmark_validation.{json,csv}`	power / benchmark validation
`results/exhibits/`	regenerated tables

7. Reproduction and verification

`reproduce.sh` performs, and exits non-zero if any stage fails:

1. **Integrity.** Each frozen stage carries a SHA-256 manifest (`manifest/`); the raw-corpus freeze rejects any capture file above the pre-set error threshold (observed non-substantive rate 0.08%, 7 of 8,640 confirmatory cells, dropped listwise).
2. **Regeneration.** The full analysis (Section 5.2) runs from the frozen inputs only.
3. **Byte-identical re-run.** Re-running `reproduce.sh` regenerates every `results/results_*.json` byte-for-byte (verified); the results of record are the committed JSON files.

Document gates. `build/build_all.sh` rebuilds every deliverable and then runs eight gates, each exiting 2 rather than 0 when it cannot check:

- anchored binding of 19 printed figures to the claim and the phrase each must follow,
- reverse coverage of all 191 manuscript literals,
- recomputation of 12 denominators, with designed and realized counts stated separately,
- byte-freshness of the two generated table bodies,
- binding of the 16 results quoted in `README.md`,
- currency of the derived blinded copy,
- integrity of the 1,365-file decontamination freeze, and
- citation resolution, which resolves every DOI at doi.org and reports an unreachable one as not checked rather than as passing.

Two-track verification. Determinism is checked from a fresh `gitclone` at the commit under test and, separately, from a capsule containing code and data only. The two catch different defects. The clone catches anything that resolves only in a full checkout, and the capsule catches a missing input, an absolute path baked into an output, and a gate that should skip rather than fail. Four runs of each track produced 28 artifacts with identical hashes within each track and across the two, and every claims file agreed leaf by leaf.

Determinism. All stochastic steps use fixed seeds (analysis seed 42; issuer-clustered bootstraps use fixed-seed resamples), so confidence intervals are exactly reproducible. *Note:* Parquet serialization is not byte-deterministic, so a rebuilt `panel.parquet` may hash differently while containing row-identical data; the confirmatory analysis reads the released frozen panel, so every reported number reproduces exactly.

Environment. Pinned analysis environment: `numpy==2.2.6`, `pandas==2.2.3`, `pyarrow==24.0.0`, `statsmodels==0.14.6`, `scipy==1.15.3`, `patsy==1.0.2`, on Python 3.10 (exact pins in `requirements.txt`). Figures are optional and need `matplotlib`, which is not part of the pinned analysis dependencies.

8. Live data collection (not part of the reproduction)

`capture/` documents how the model-response corpora were originally collected: `collect.py` drives the main confirmatory grid; `run_arm.py` and `realarm_collect.py` run the real-firm arm; `run_flagship.py` runs the flagship model-tier arm and `freeze_flagship.py` writes its SHA-256 manifest; `run_reasoning.py` runs the reasoning-architecture arm (DeepSeek-R1 via OpenRouter) and `freeze_reasoning.py` writes `MANIFEST_REASONING`; `fingerprint_pilot.py` runs the perturbation-and-fingerprint-screening gate; `list_models.py` enumerates served models. This phase requires internet access and provider API keys, incurs cost, and is excluded from the reproducible core. The exclusion covers the collection scripts, not the whole directory: `capture/flagship_runs/` and `capture/reasoning_runs/` hold frozen corpora that the analysis reads, and `capture/freeze_decontam.py` is imported by the decontamination analysis. API keys are never distributed (`.gitignore` blocks `keys.env.txt` and any `API Keys/` folder); all reported results derive only from the frozen corpora.

9. Licensing and data availability

Code is released under the MIT Licence (see `LICENSE`); data, analysis outputs, and text are released under CC-BY-4.0. The confirmatory data are synthetic; the real-firm arm uses only public SEC filings (anonymized and perturbed). The permanent citable archive is the Zenodo deposit at `10.5281/zenodo.21953934`, which reproduces every reported number, table, and figure offline, deterministically, and at zero cost.